

Hugging Face LeRobot

examples/phone_to_so100 Complete Source Code (Tablet Edition)

Repository: <https://github.com/huggingface/lerobot>

Directory: examples/phone_to_so100

Target Robot: SO-100 / SO-101 Follower Arm

Teleoperator: Smartphone (iOS ARKit / Android Sensors via LeRobot Phone Teleoperator)

Display Settings: Large 9.6pt Font, Slim Margins (4mm top/bottom, 5mm left/right), Section-by-Section
Pagination

Table of Contents



1. [teleoperate.py](#) 목표

Real-time teleoperation test script using smartphone motion to control the SO-100 robot arm. (106 lines)

2. [record.py](#)

Record demonstration episodes (phone actions, camera video feeds, robot joint observations) into a LeRobot dataset. (200 lines)

3. [replay.py](#)

Open-loop replay of recorded End-Effector trajectories from a LeRobot dataset onto the physical robot via IK. (90 lines)

4. [rollout.py](#)

Autonomous execution of a trained imitation learning policy (e.g., ACT) on the robot in EE space without phone. (110 lines)

5. [evaluate.py](#) → 정책 평가 및 벤치마크

Full evaluation loop for trained policies across multiple episodes with logging, rerun visualization, and dataset recording. (221 lines)

다른 예제에서도 동일한 것 같아 일단 같이 공부

```
1 import time
2
3 from lerobot.lerobot_types import RobotAction, RobotObservation
4 from lerobot.model.kinematics import RobotKinematics
5 from lerobot.processor import (
6     RobotProcessorPipeline,
7     robot_action_observation_to_transition,
8     transition_to_robot_action,
9 )
10 from lerobot.robots.so_follower import S0100Follower, S0100FollowerConfig
11 from lerobot.robots.so_follower.robot_kinematic_processor import (
12     EEBoundsAndSafety,
13     EEReferenceAndDelta,
14     GripperVelocityToJoint,
15     InverseKinematicsEEToJoints,
16 )
17 from lerobot.teleoperators.phone import Phone, PhoneConfig
18 from lerobot.teleoperators.phone.config_phone import PhoneOS
19 from lerobot.teleoperators.phone.phone_processor import MapPhoneActionToRobotAction
20 from lerobot.utils.robot_utils import precise_sleep
21 from lerobot.utils.visualization_utils import init_rerun, log_rerun_data
22
23 FPS = 30
```

fps 설정

```

26 def main():
27     # Initialize the robot and teleoperator
28     robot_config = SO100FollowerConfig(
29         port="/dev/tty.usbmodem5A460814411", id="my_awesome_follower_arm", use_degrees=True
30     )
31     teleop_config = PhoneConfig(phone_os=PhoneOS.IOS) # or PhoneOS.ANDROID
32
33     # Initialize the robot and teleoperator
34     robot = SO100Follower(robot_config)
35     teleop_device = Phone(teleop_config)
36
37     # NOTE: It is highly recommended to use the urdf in the SO-ARM100 repo: https://github.com/TheRobotStudio/SO-ARM100/blob/main/Simulation/SO101/so101_new_calib.urdf
38     kinematics_solver = RobotKinematics(
39         urdf_path="/SO101/so101_new_calib.urdf",
40         target_frame_name="gripper_frame_link",
41         joint_names=list(robot.bus.motors.keys()),
42     )
43
44     # Build pipeline to convert phone action to ee pose action to joint action
45     phone_to_robot_joints_processor = RobotProcessorPipeline[
46         tuple[RobotAction, RobotObservation], RobotAction
47     ](
48         1. steps=[
49             MapPhoneActionToRobotAction(platform=teleop_config.phone_os),
50             EEReferenceAndDelta(
51                 kinematics=kinematics_solver,
52                 end_effector_step_sizes={"x": 0.5, "y": 0.5, "z": 0.5},
53                 motor_names=list(robot.bus.motors.keys()),
54                 use_latched_reference=True,
55             ),
56             EEBoundsAndSafety(
57                 end_effector_bounds={"min": [-1.0, -1.0, -1.0], "max": [1.0, 1.0, 1.0]},
58                 max_ee_step_m=0.10,
59             ),
60             GripperVelocityToJoint(
61                 speed_factor=20.0,
62             ),
63             InverseKinematicsEEToJoints(
64                 kinematics=kinematics_solver,
65                 motor_names=list(robot.bus.motors.keys()),
66                 initial_guess_current_joints=True,
67             ),
68         ],
69         2. to_transition=robot_action_observation_to_transition,
70         3. to_output=transition_to_robot_action,
71     )

```

판로워 요 선정
 핸드폰 설정
 Kinematic solver는 아래 연산 수행
 EE 이름 선정
 Kinematics = 기구학 (힘과 질량 무시, 기하학만으로 움직임을 다루는 학문)
 Forward Kinematics: 관절 각도 → 그리퍼의 위치와 자세 구하기
 Inverse Kinematics: 그리퍼 목표 위치 → 관절 각도 구하기
 urdf: 로봇의 링크(뼈) 길이, 관절 축 등을 담은 xml
 python 문법 → 객체이름 [대체 변수] (생성과 함수)
 ~ const int xp
 입력타입
 출력타입
 MapPhoneActionToRobotAction(platform=teleop_config.phone_os), → 핸드폰 데이터 처리
 EEReferenceAndDelta(변화량
 kinematics=kinematics_solver,
 end_effector_step_sizes={"x": 0.5, "y": 0.5, "z": 0.5}, → 끝 움직임 당 EE 움직임 비율
 motor_names=list(robot.bus.motors.keys()),
 use_latched_reference=True, → 핸드폰의 위치를 상대 좌표로
 1 = 핸드폰과 동일
 0.5 = 핸드폰의 1/2 만큼
 EE 이동량 변환
 EE 안전장치
 EE 안전장치
 GripperVelocityToJoint(
 speed_factor=20.0, → 실제 움직임(잡기)의 민감도
 InverseKinematicsEEToJoints(
 kinematics=kinematics_solver,
 motor_names=list(robot.bus.motors.keys()),
 initial_guess_current_joints=True, → 실제로 측정된 현재 관절 각도로 IK 계산
 False: 이전 프레임에서 IK가 계산한 값으로 → replay 때 큰 사용
 2. to_transition=robot_action_observation_to_transition,
 3. to_output=transition_to_robot_action, → EnvTransition 변환

EnvTransition 란?

· 파이썬에서의 표준 딥러닝 형식

```

EnvTransition = {
    "observation": { ... },
    "action": { ... },
    "reward": { ... },
    "done": { ... },
    ...
}

```

센서 데이터 전체
 실행한 (실행된) 액션 → 로봇이 움직일 때 필요한 유인 데이터
 강화학습 보상
 에피소드 종료 여부
 = output에 action만 담음 (변환 사용)


```

73 # Connect to the robot and teleoperator
74 robot.connect()  로봇 연결
75 teleop_device.connect()  핸드폰 연결
76
77 # Init rerun viewer
78 init_rerun(session_name="phone_so100_teleop")  rerun 세션 이름 설정
79
80 if not robot.is_connected or not teleop_device.is_connected:  로봇, 핸드폰 연결 체크
81     raise ValueError("Robot or teleop is not connected!")
82
83 print("Starting teleop loop. Move your phone to teleoperate the robot...")
84 while True:
85     t0 = time.perf_counter()  ← 정밀 스톱워치
86
87     # Get robot observation
88     robot_obs = robot.get_observation() = 로봇의 값
89     # Get teleop action
90     phone_obs = teleop_device.get_action() = 핸드폰의 값
91
92     # Phone -> EE pose -> Joints transition
93     joint_action = phone_to_robot_joints_processor((phone_obs, robot_obs))
94     # 위에 있던 객체
95
96     # Send action to robot
97     _ = robot.send_action(joint_action)
98     # 반환값이 있지만 의도적으로 안쓴다를 표현 / 없어도 기능은 동일
99     # Visualize
100    log_rerun_data(observation=phone_obs, action=joint_action) => rerun
101
102    precise_sleep(max(1.0 / FPS - (time.perf_counter() - t0), 0.0)) => 일정한 주기로 돌아가게 하는 식
103    (time.sleep()보다 정확)  ↳ 정해진 시간만큼 쉬기  ↳ 한 프레임의 시간  ↳ EO에서 리금까지 걸린 시간  ↳ (목표 시간 - 실제 걸린 시간) 만큼 쉬기
104
105 if __name__ == "__main__":
106     main()

```

phone_to_robot_joints_processor

call 함수

1. self.to_transition(data) 입력을 Env Transition 포맷
2. self.forward(transition) step 실행
3. self.to_output(transformed_transition) 최종 결과물

--XX-- ← 더더 (dunder) : 특정 상황에 자동 호출

단 __main__ 은 매서드가 아닌 변수

if __name__ == "__main__" ← 이 파일을 직접 실행하면 true
import 해서 사용하면 false

왜 일률로 쉬게 하나?

1. 폰(센서)의 데이터 전달 한계
2. 로봇모터 움직임의 한계
3. sleep이 없으면 해당 스레드가 CPU 100% 점유

↳ 그럼 언제 못 쉬게?

최대한의 반응 지연을 줄일 때